

Find the candidate keys of a relation, How to find the candidate keys, Which is the key for the given table, concept of candidate key in dbms, candidate key examples

Question:

Consider the relation $R = \{A, B, C, D, E, F, G, H, I, J\}$ and the set of functional dependencies $F = \{AB \rightarrow C, A \rightarrow DE, B \rightarrow F, F \rightarrow GH, D \rightarrow IJ\}$. Find the key of relation R.

Let $R = (A, B, C, D, E, F)$ be a relation scheme with the following dependencies-

$$C \rightarrow F$$

$$E \rightarrow A$$

$$EC \rightarrow D$$

$$A \rightarrow B$$

Which of the following is a key for R?

Third Normal Form

Definition:

- **Transitive functional dependency:** a FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$

Examples:

- $SSN \rightarrow DMGRSSN$ is a **transitive** FD
 - Since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ hold
- $SSN \rightarrow ENAME$ is **non-transitive**
 - Since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow ENAME$

Third Normal Form

A relation schema R is in **third normal form (3NF)** if it is in 2NF *and no non-prime attribute A in R is transitively dependent on the primary key.*

R can be decomposed into 3NF relations via the process of 3NF normalization

NOTE:

- In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.
- When Y is a candidate key, there is no problem with the transitive dependency .
- E.g., Consider EMP (SSN, Emp#, Salary).
 - Here, $SSN \rightarrow Emp\#$, $Emp\# \rightarrow Salary$ and Emp# is a candidate key.

Normal Forms Defined Informally

1st normal form

- All attributes depend on **the key**

2nd normal form

- All attributes depend on **the whole key**

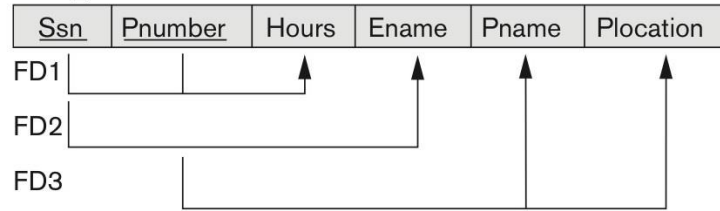
3rd normal form

- All attributes depend on **nothing but the key**

Normalizing into 2NF and 3NF

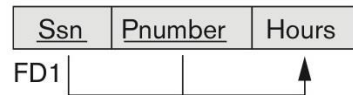
(a)

EMP_PROJ

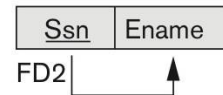


2NF Normalization

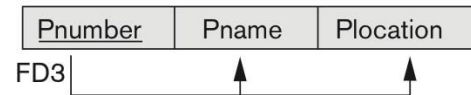
EP1



EP2



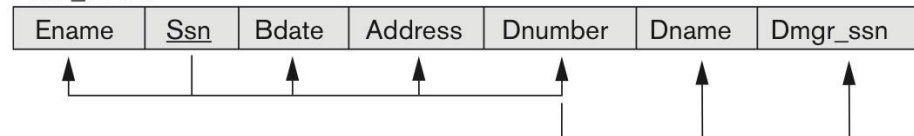
EP3



Normalizing into 2NF and 3NF.
(a) Normalizing EMP_PROJ into 2NF relations.
(b) Normalizing EMP_DEPT into 3NF relations.

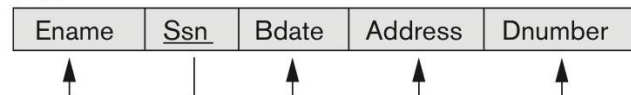
(b)

EMP_DEPT

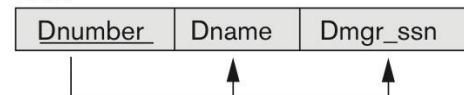


3NF Normalization

ED1



ED2



Normalization into 2NF and 3NF

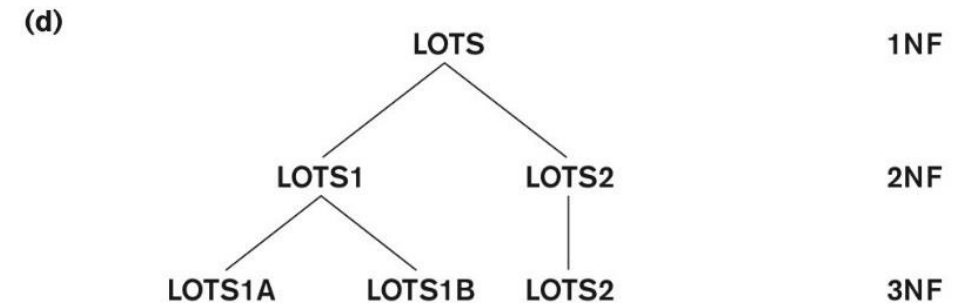
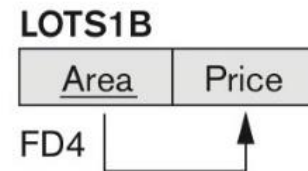
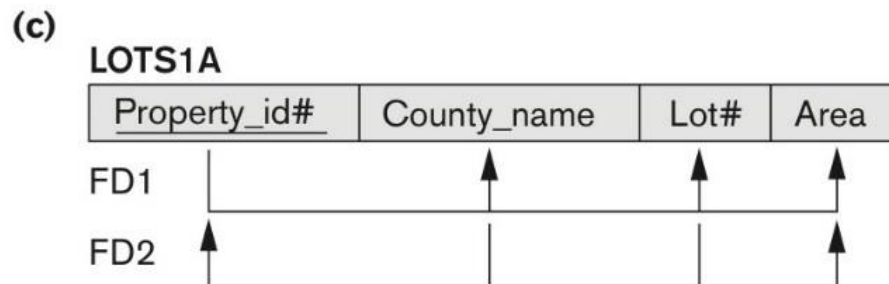
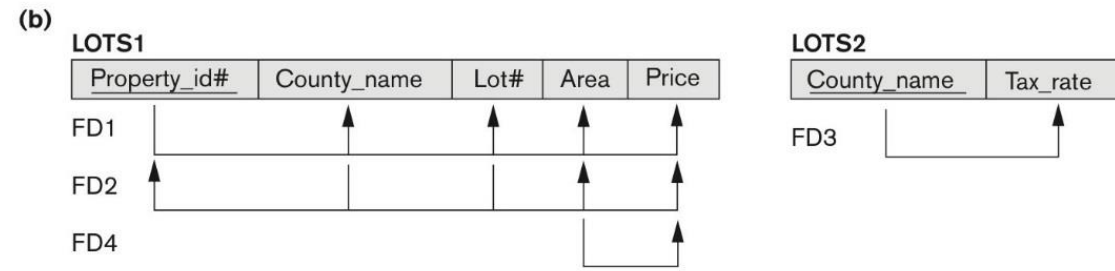
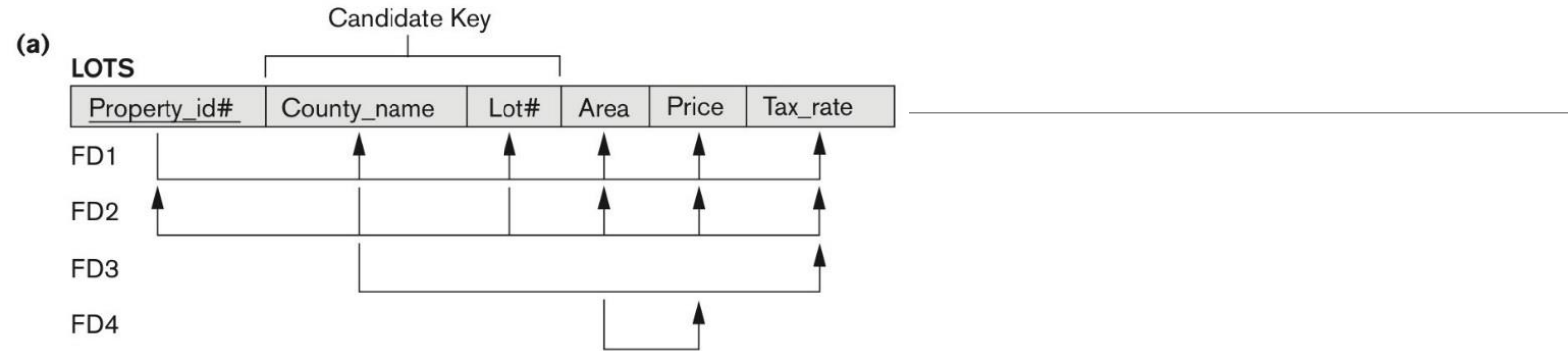
Normalization into 2NF and 3NF.

(a) The LOTS relation with its functional dependencies FD1 through FD4.

(b) Decomposing into the 2NF relations LOTS1 and LOTS2.

(c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B

(d) Progressive normalization of LOTS into a 3NF design.



General Normal Form Definitions (For Multiple Keys)

The above definitions consider the primary key only

The following more general definitions take into account relations with multiple candidate keys

Any attribute involved in a candidate key is a prime attribute

All other attributes are called non-prime attributes.

General Definition of Third Normal Form

Definition:

- **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
- A relation schema R is in **third normal form (3NF)** if whenever a FD, $X \rightarrow A$ holds in R, then either:
 - (a) X is a superkey of R, or
 - (b) A is a prime attribute of R

LOTS1 relation violates 3NF because

Area $\rightarrow$ Price ; and Area is not a superkey in LOTS1.

Interpreting the General Definition of Third Normal Form

Consider the 2 conditions in the Definition of 3NF:

A relation schema R is in **third normal form (3NF)** if whenever a FD $X \rightarrow A$ holds in R , then either:

- (a) X is a superkey of R , or
- (b) A is a prime attribute of R

Condition (a) catches two types of violations :

- one where a prime attribute functionally determines a non-prime attribute.

This catches 2NF violations due to non-full functional dependencies.

-second, where a non-prime attribute functionally determines a non-prime attribute. This catches 3NF violations due to a transitive dependency.

Interpreting the General Definition of Third Normal Form

ALTERNATIVE DEFINITION of 3NF: We can restate the definition as:

A relation schema R is in **third normal form (3NF)** if every non-prime attribute in R meets both of these conditions:

- It is fully functionally dependent on every key of R
- It is non-transitively dependent on every key of R

Note that stated this way, a relation in 3NF also meets the requirements for 2NF.

The condition (b) from the last slide takes care of the dependencies that “**slip through**” (are allowable to) **3NF** but are “caught by” BCNF.

BCNF (Boyce-Codd Normal Form)

A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD** $X \rightarrow A$ holds in R , then **X is a superkey** of R

Each normal form is strictly stronger than the previous one

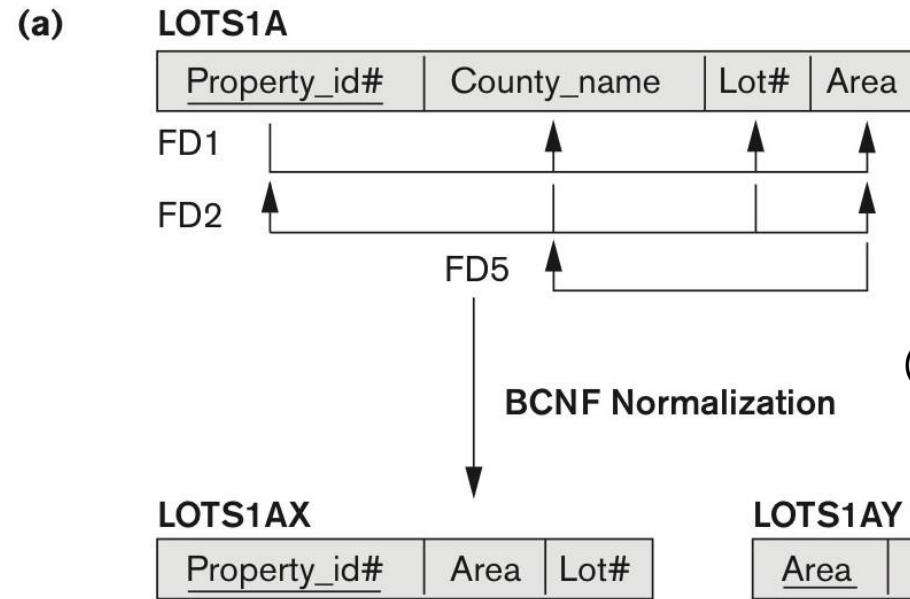
- Every 2NF relation is in 1NF
- Every 3NF relation is in 2NF
- Every BCNF relation is in 3NF

There exist relations that are in 3NF but not in BCNF

Hence BCNF is considered a **stronger form of 3NF**

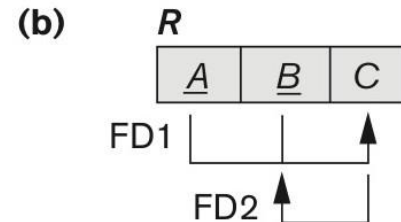
The goal is to have each relation in BCNF (or 3NF)

Boyce-Codd normal form



Boyce-Codd normal form.
 (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition.

(b) (b) A schematic relation with FDs; it is in 3NF, but not in BCNF due to the f.d. $C \rightarrow B$.



A relation TEACH that is in 3NF but not in BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

A relation TEACH that is in
3NF but not BCNF.

Achieving the BCNF by Decomposition

Two FDs exist in the relation TEACH:

- fd1: { student, course } $\rightarrow$ instructor
- fd2: instructor $\rightarrow$ course

{student, course} is a candidate key for this .

So this relation is in 3NF *but not in* BCNF.

A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.

Decompose into 2NF and 3NF relations

Consider the universal relation $R = \{A, B, C, D, E, F, G, H, I, J\}$ and the set of functional dependencies $F = \{\{A, B\} \rightarrow \{C\}, \{A\} \rightarrow \{D, E\}, \{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}, \{D\} \rightarrow \{I, J\}\}$. What is the key for R ? Decompose R into 2NF and then 3NF relations.

Thank you